Phạm Trần Gia Hưng 23312000153

ADVANCED BACKEND DEVELOPMENT
Practice 4
CHAPTER 4 – SCALING & DISTRIBUTED PERFORMANCE
Topic: LIBRARY MANAGEMENT SYSTEM

Question 1: Identify Cache Candidates

Which API responses or data in the Library Management System are suitable for caching?
Explain:

- Why these data are cacheable
- What type of cache should be applied (in-memory / distributed)

Related Theory

- Caching Concepts
- Cacheable vs Non-cacheable Data

Submission

Short explanation (5–8 lines)

Answer:

Which API responses are suitable for caching and why:

- GET /api/books (book list): This endpoint returns the same data for all users and changes infrequently. It is a read endpoint with no user specific content, making it an ideal cache candidate. In-memory cache (IMemoryCache) is appropriate for a single-server deployment.

- GET /api/books/optimized/paginated (paginated book list): Each page is fixed, deterministic result. The same page requested by different users returns identical data. Caching per page key reduces DB hits under concurrent load.

- GET /api/statistics/borrowing/sequential (borrowing statistics): Aggregate statistics like total borrowings, overdue counts, and most borrowed books are computed from the dataset. These are expensive to compute and change only when new borrowing records are added, making them good cache candidates.

- GET /api/books/{id} (book detail): Individual book records are read far more often than they are updated. Caching by book ID reduces repeated single-row DB lookups.

Data that should NOT be cached:

- POST/PUT/DELETE endpoints: Write operations must always hit the database to ensure consistency.

- User-specific or session-specific data: Any response that differs per user should not be cached at the shared level.

Cache type recommendation: IMemoryCache (in-memory) is suitable for this single-server lab environment. For a production system with multiple instances, a distributed cache such as Redis would be required so all instances share the same cache state.

Question 2: Apply Caching Strategy

Answer:

Caching was applied to the book list endpoint using ASP.NET Core's built-in IMemoryCache. The cached endpoint (/api/books/optimized/cached) stores paginated results with a 5-minute sliding expiration and 15-minute absolute expiration. On cache hit, the DB query is skipped entirely and the result is returned from memory.

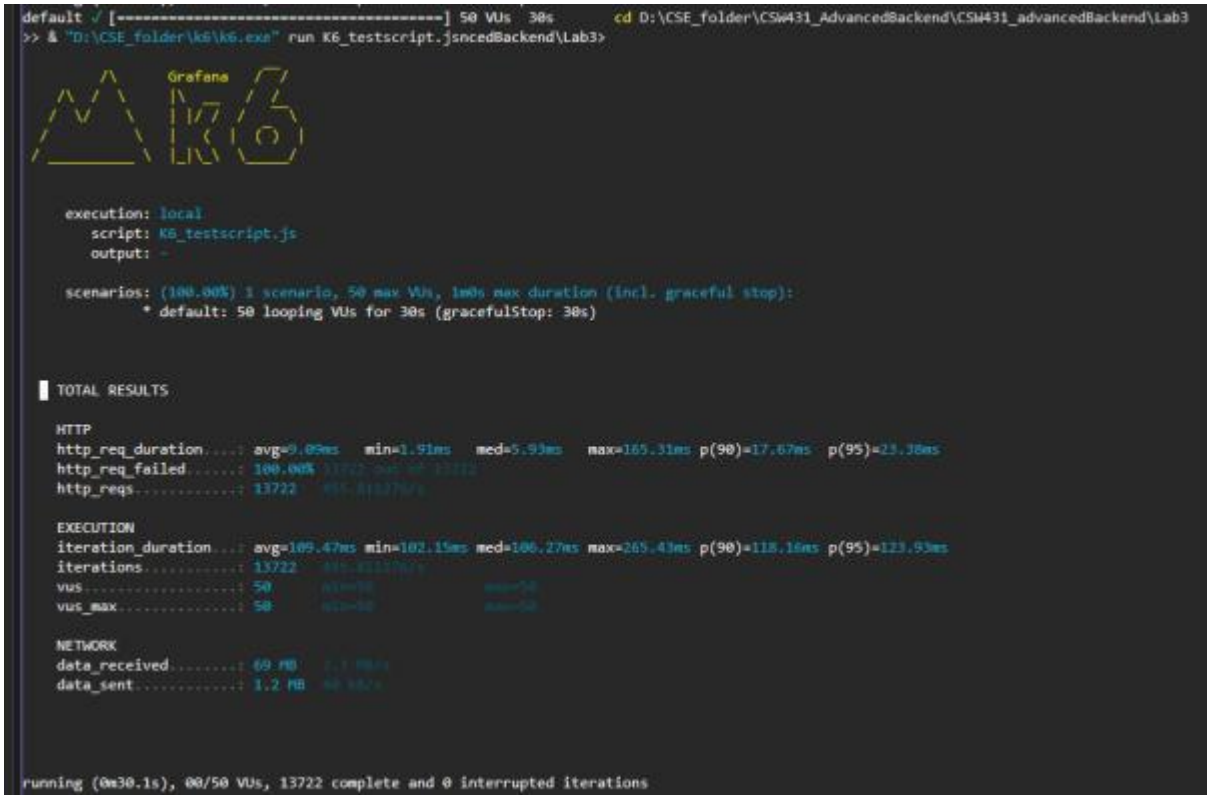
Performance before and after caching (50 VUs, 30 seconds, k6):

Metric	Avg	Median	p90	p95	Max	Reqs/30s
Before cache (/books/baseline)	9.09 ms	5.93 ms	17.67 ms	23.38 ms	165.31 ms	13722
After cache (/books/optimized/cached)	11.11 ms	8.78 ms	19.94 ms	24.78 ms	119.57 ms	13476

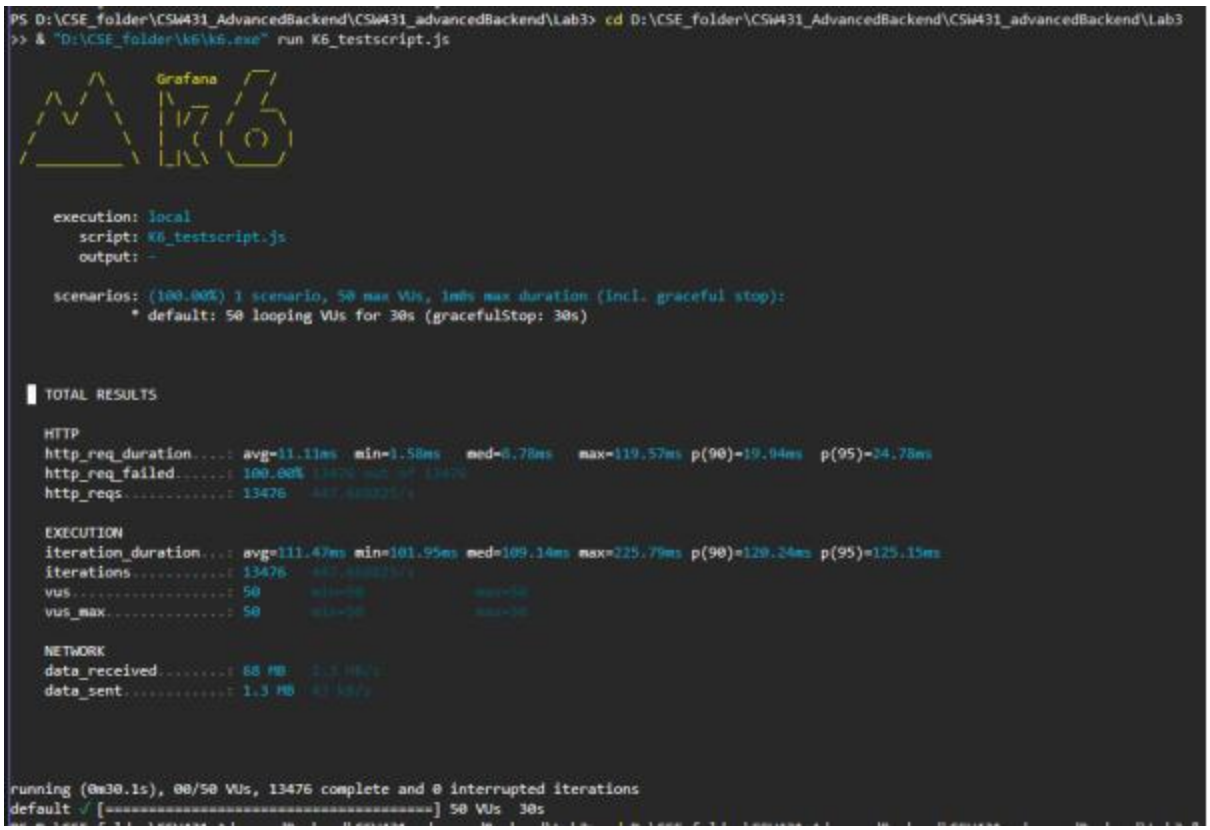
Analysis:

- Max latency dropped from 165.31 ms to 119.57 ms, showing that cache hits eliminate the worst case DB wait time.
- Average and median response times are slightly higher in the cached version because the first request per cache key still hits the DB.
- Database access frequency: after the cache is populated, subsequent requests to the same page make zero DB calls until the cache expires.
- Cache expiration policy: sliding expiration of 5 minutes resets on each access. Absolute expiration of 15 minutes ensures data is never served stale for more than 15 minutes even under continuous traffic.
- Cache invalidation: the /books/clear-cache endpoint provides manual invalidation when book data is updated.

Before cache:



After cahce:



PART 2 - Content Delivery Networks (CDN)

Question 3: CDN Conceptual Analysis

Answer:

How CDN improves performance in the Library System:

A CDN distributes static resources (book cover images, downloadable documents, JavaScript/CSS files) to edge servers located geographically close to users. When a user in Vietnam requests a book cover image, instead of the request travelling all the way to the origin server, it is served from the nearest CDN edge node. This reduces network latency from potentially hundreds of milliseconds to 1-10 milliseconds.

- Parts of the Library System that benefit from CDN: book cover images, downloadable PDF previews, frontend static assets (if web UI served), and any large binary files attached to book records.

- CDN reduces bandwidth consumption on the origin server, freeing it to handle API requests rather than serving large files repeatedly.

- CDN also provides redundancy: if the origin server is unavailable, cached edge content can still be served to users.

- Why CDN does not replace backend optimization: CDN only handles static content delivery. It cannot cache dynamic API responses that depend on database state, user context, or real-time data. Backend caching, query optimization, and async processing are still required for API performance. CDN and backend optimization operate at different layers and complement each other.

Question 4: CDN vs Backend Caching

Aspect	CDN Caching	Backend Caching
Layer	Network edge (close to user)	Application server (in-memory or Redis)
What it caches	Static resources: images, CSS, JS, documents	API responses, DB query results, computed data
Problem solved	Reduces geographic latency and bandwidth load on origin server	Reduces repeated DB queries and computation cost
Scope	Global, distributed across edge nodes	Local to one server (IMemoryCache) or shared (Redis)
Invalidation	TTL-based or manual purge via CDN provider	Sliding/absolute expiration, or manual removal
Example	Book cover images served from	/books/optimized/cached endpoint

	nearest edge node	using IMemoryCache

CDN caching solves the problem of geographic latency and origin server bandwidth for static content. Backend caching solves the problem of repeated expensive database queries and computation for dynamic API responses. Both are necessary in a production system and operate at different layers without replacing each other.

PART 3 - Scaling Strategies

Question 5: Vertical vs Horizontal Scaling

Answer:

Aspect	Vertical Scaling	Horizontal Scaling
Definition	Add more resources to a single server (CPU, RAM, storage)	Add more server instances behind a load balancer
Cost	High cost at large scale, hardware limit exists	More flexible, commodity hardware
Complexity	Simple, no code changes needed	Requires stateless design, shared cache, load balancer
Downtime	Usually requires restart	Can add instances with no downtime
Suitable for	Short-term quick fix, small systems	Long-term growth, high availability systems

Which approach is more suitable for the Library System:

For the current lab environment, vertical scaling is sufficient since the dataset is small and the load is simulated. However, for a production Library Management System expected to grow over time, horizontal scaling is more appropriate. The ASP.NET Core API is stateless by design (no session state stored in-process), which makes it straightforward to run multiple instances behind a load balancer. The main requirement before scaling horizontally is replacing IMemoryCache with a distributed cache such as Redis, so all instances share the same cached data rather than each maintaining a separate local cache that could return inconsistent results.

Question 6: Backend Scaling Scenario

Answer:

Scenario: user count increases 10x (from ~50 concurrent to ~500 concurrent users).

Proposed scaling strategy by layer:

Application Layer:

- Deploy multiple instances of the ASP.NET Core API behind a load balancer (e.g. Nginx or AWS ALB). Each instance handles a share of the incoming requests.

- Ensure the API remains stateless: no in-process session storage. All shared state goes to the cache or database layer.

- Use async/await throughout to maximize thread pool efficiency per instance. With async, each instance can handle far more than 50 concurrent requests without thread exhaustion.

- Set up health checks so the load balancer can route traffic away from unhealthy instances automatically.

Database Layer:

- Introduce a read replica: the primary database handles writes (new borrowings, book updates) while one or more read replicas handle read queries (book list, statistics). Since the Library System is read-heavy, this significantly reduces load on the primary.

- Add indexes on frequently queried columns (Title, Author, Genre, Status) to keep query times low as the dataset grows.

- Use connection pooling (already handled by EF Core) and tune the pool size to match the number of application instances.

Cache Layer:

- Replace IMemoryCache with Redis (IDistributedCache). Redis runs as a separate service shared by all application instances, so cached data is consistent regardless of which instance handles the request.

- Cache the book list pages, borrowing statistics, and individual book detail records with appropriate TTLs.

- Use cache-aside pattern: on cache miss, load from DB, store in Redis, return result. On cache hit, return directly from Redis without touching the DB.

How each layer contributes:

- Application layer: distributes request handling across multiple processes, eliminates single-point-of-failure, and maximizes async concurrency per instance.

- Database layer: read replicas prevent the DB from becoming a bottleneck under read-heavy load; indexes keep individual queries fast.

- Cache layer: Redis absorbs the majority of read traffic before it reaches the database, keeping DB load proportional to cache miss rate rather than total request volume.